Project Portfolio – Noa Žuvela

Autonomous Robot-Based Testing System | Master's Thesis

Danfoss Power Solutions · University of Southern Denmark · 2026

Designed and validated a reconfigurable end-of-line testing station for industrial joysticks. A UR3 robot uses real-time computer vision and stereo depth to locate buttons, a locally deployed language model to plan press sequences, and force/torque plus CAN-bus feedback to verify physical and electrical functionality.

3	248	100%
joystick profiles tested	**test sequences** completed	**task completion** across all runs

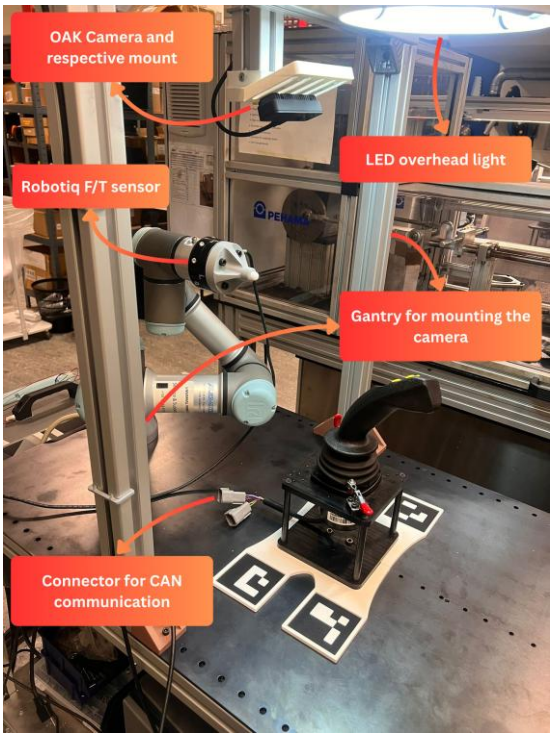
The finished test station

CHALLENGE Conventional joystick testers depend on fixed fixtures and manually maintained programs when button layouts change.

SOLUTION A modular vision-to-action system detects control positions, creates robot-executable targets, and runs the complete press sequence through a single interface.

MY ROLE End-to-end design and implementation: mechanical test station, CAD and fabrication, camera/robot calibration, Python software, perception, planning, robot communication, GUI, and validation.

FLEXIBILITY New joystick profiles require an initial setup; saved profiles can then be switched without per-session recalibration or changes to robot motion scripts.



Built workstation: UR3, OAK-D camera, F/T sensor, lighting and CAN connector.

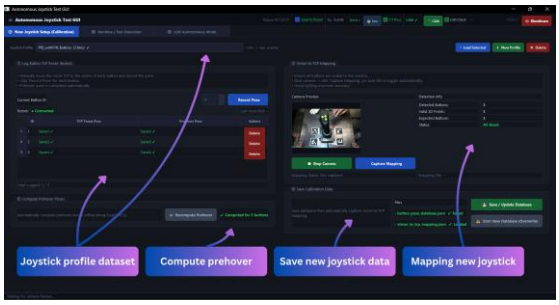
How a test is executed

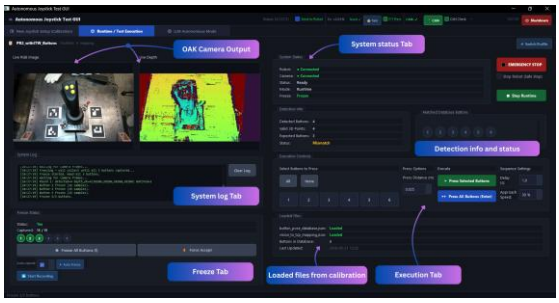
01 Detect **02 Localise** **03 Stabilise** **04 Plan** **05 Execute & verify**

Hardware and software: Universal Robots UR3 / URScript / RTDE · OAK-D + DepthAI · YOLOv8n / OpenCV · PyQt5 / Python · Ollama (Qwen2.5 1.5B) · Robotiq FT300 · Vector VN1610 / CAN

Final software interface | Three operating modes

The PyQt5 application connects calibration, live perception, robot execution, autonomous task planning and validation in one operator-facing system.





01 CALIBRATION

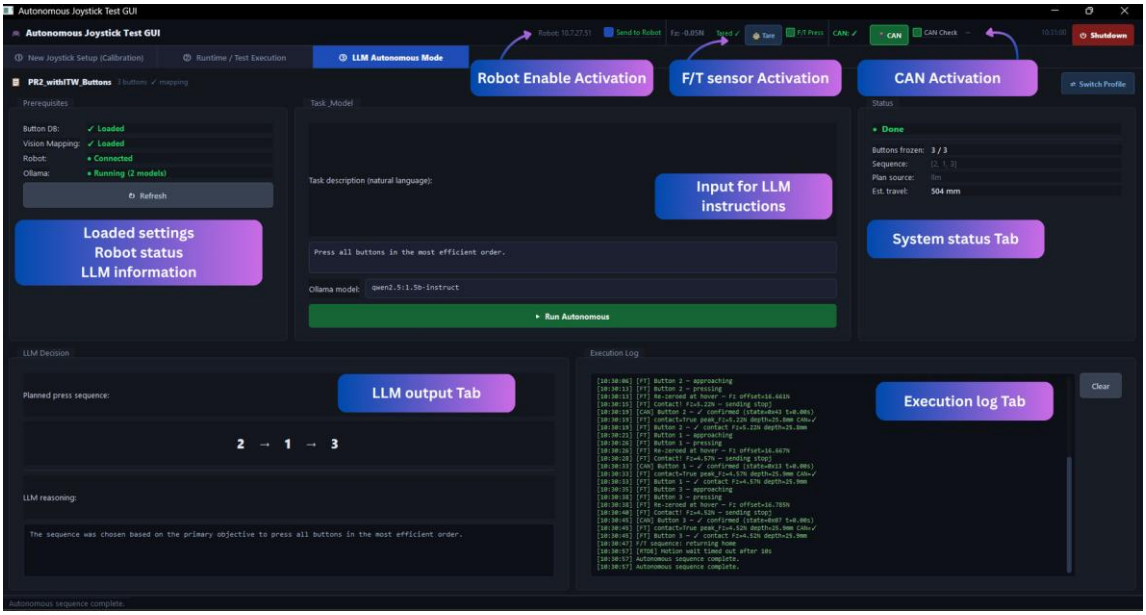
Record TCP button positions, compute pre-hover poses and save per-joystick vision mapping.

02 RUNTIME / TEST EXECUTION

Inspect RGB/depth feeds, stabilise button positions and run manual selections.

03 Autonomous language-model mode

A local Qwen2.5 1.5B model interprets the task description and proposes a structured button order; validation and a deterministic fallback keep execution constrained to known button IDs.



Autonomous-mode screenshot: task prompt, generated sequence (2 → 1 → 3), system prerequisites and execution log.

The final interface also displays live robot, F/T, and CAN statuses, per-button test events, and operator-accessible stop controls.

Experimental validation | Results & engineering findings

99.48%	**95.2–100%**	**98.4–98.8%**
mAP@0.5	**F/T contact**	**CAN confirmation**
YOLOv8n test set	by joystick profile	wired profiles
62 / 62	**100%**	**5–15 ms**
valid test plans	**freeze completion**	**visual inference**
with guarded fallbacks	tested sessions	per image on GPU

240 mechanically confirmed, electrically silent press events detected

On a deliberately disconnected joystick, the system detected physical actuation through the F/T sensor but correctly flagged the absence of the expected CAN signal.

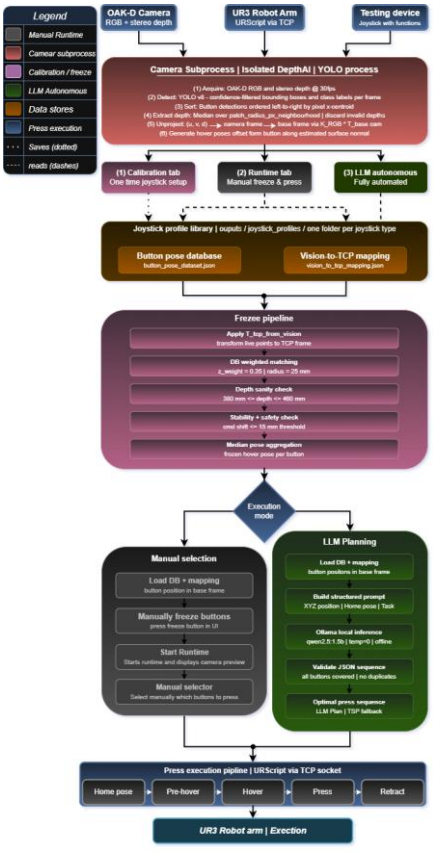
Key engineering contributions

Robust perception. Trained YOLOv8n on 620 annotated real and CAD-rendered images and developed multi-frame median-based depth stabilisation to cope with approximately 33% per-frame depth dropout.

Persistent calibration. Integrated ArUco/ChArUco calibration and SVD-based vision-to-robot mapping. Saved profile data support repeated sessions and fast switching between registered variants.

Local, guarded planning. Used a locally running small language model for task-dependent button ordering, with JSON validation and deterministic travelling-salesman fallback.

Closed-loop validation. Combined adaptive F/T-triggered pressing with CAN electrical monitoring, separating physical contact from a true electrical response.



Complete architecture and data flow

Outcome and scope

Across 248 laboratory test sequences and three configured joystick profiles, the system completed every task and demonstrated automated failure identification using two independent sensing channels. The vision model was trained on the PR2 button family, and the current prototype assumes controlled lighting and a fixed camera/fixture geometry; wider variant generalisation and production deployment remain future work.

Roll-to-Roll Droplet Deposition & AI Monitoring System

University research project focused on the automation and monitoring of a roll-to-roll (R2R) material deposition system. The objective was to develop a system capable of dispensing droplets at controlled sizes and rates in a predefined grid pattern, while enabling real-time feedback on droplet shape, size, and placement during operation.

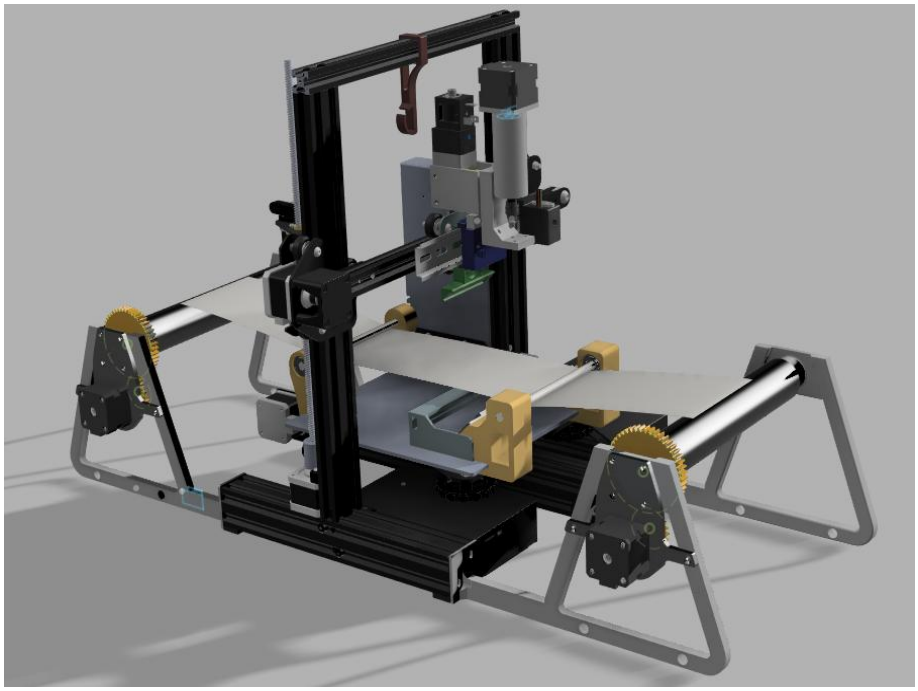


Figure 5: R2R Full Assembly

Role

Focused on the development of the droplet detection principle and the AI-based detection system. In addition, contributed across multiple aspects of the project, including system design, additive manufacturing of components, and programming, supporting overall development and integration of the platform.

Technical Contributions

- Technologies: Arduino, Embedded Systems, Computer Vision, Python, Image Processing, Mechanical Design
- Development of embedded control system using Arduino for low-level actuation and timing-critical operations
- Implementation of synchronized motion control between subsystems to ensure consistent and repeatable operation
- Design and implementation of a computer vision pipeline based on optical distortion detection for identifying transparent droplets

- Design and fabrication of a custom experimental test setup, including camera positioning, controlled illumination, and calibration geometry
- Data collection, model training, and validation of detection performance under controlled experimental conditions

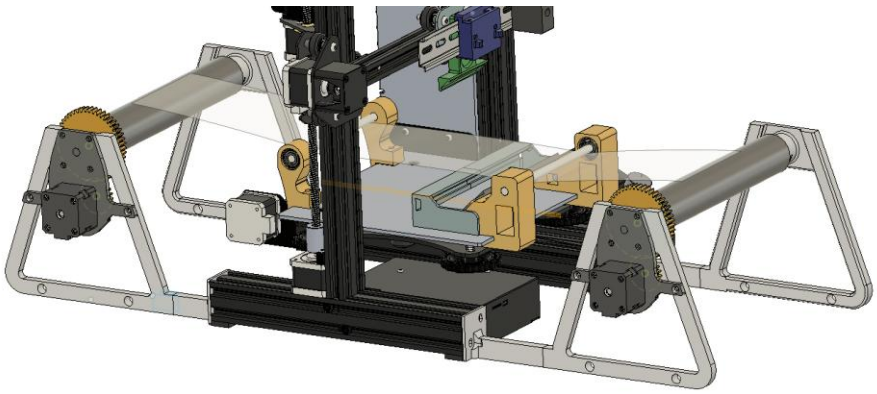


Figure 6: R2R mechanism

Outcome

Developed a real-time droplet detection and monitoring system enabling precise control and feedback in automated deposition processes, improving process reliability and supporting scalable automation of material manufacturing systems. The system enables real-time quality monitoring through vision-based feedback integration.



Figure 7: Droplet detection

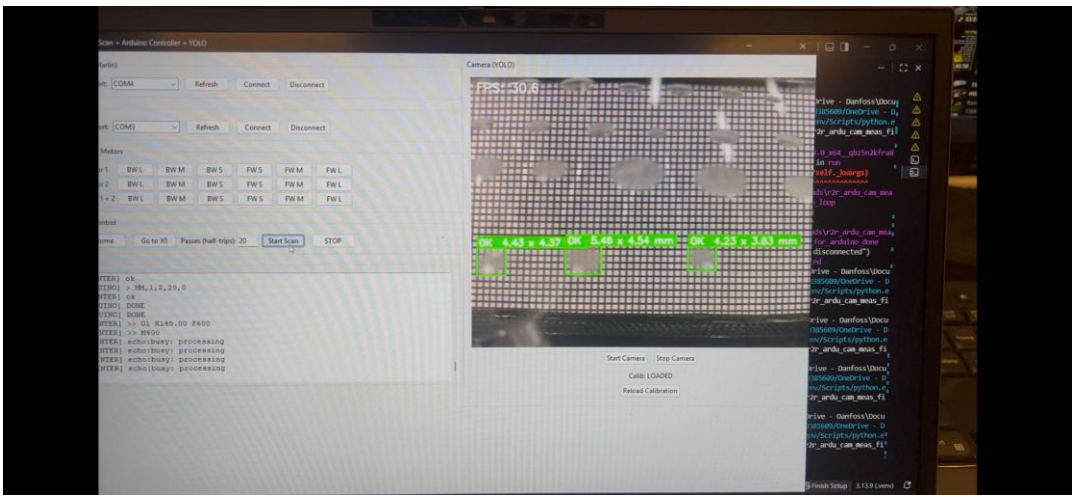


Figure 8: User interface for droplet detection

Hand Gesture Recognition using CNNs

Developed a computer vision model for classifying hand gestures and rejecting irrelevant inputs, simulating real-world human-computer interaction scenarios.



Figure 9: Hand gesture data

Role

Designed and implemented the full machine learning pipeline including data preprocessing, model training, and evaluation. Applied transfer learning and optimization techniques to improve robustness and generalization.

Technical Contributions

- Implemented transfer learning pipeline using pretrained ResNet18 in PyTorch
- Designed data augmentation pipeline to improve generalization and robustness
- Implemented label smoothing and weighted sampling
- Fine-tuned selected layers for improved performance
- Designed training pipeline with learning rate scheduling
- Evaluated model performance using confusion matrices and training dynamics analysis

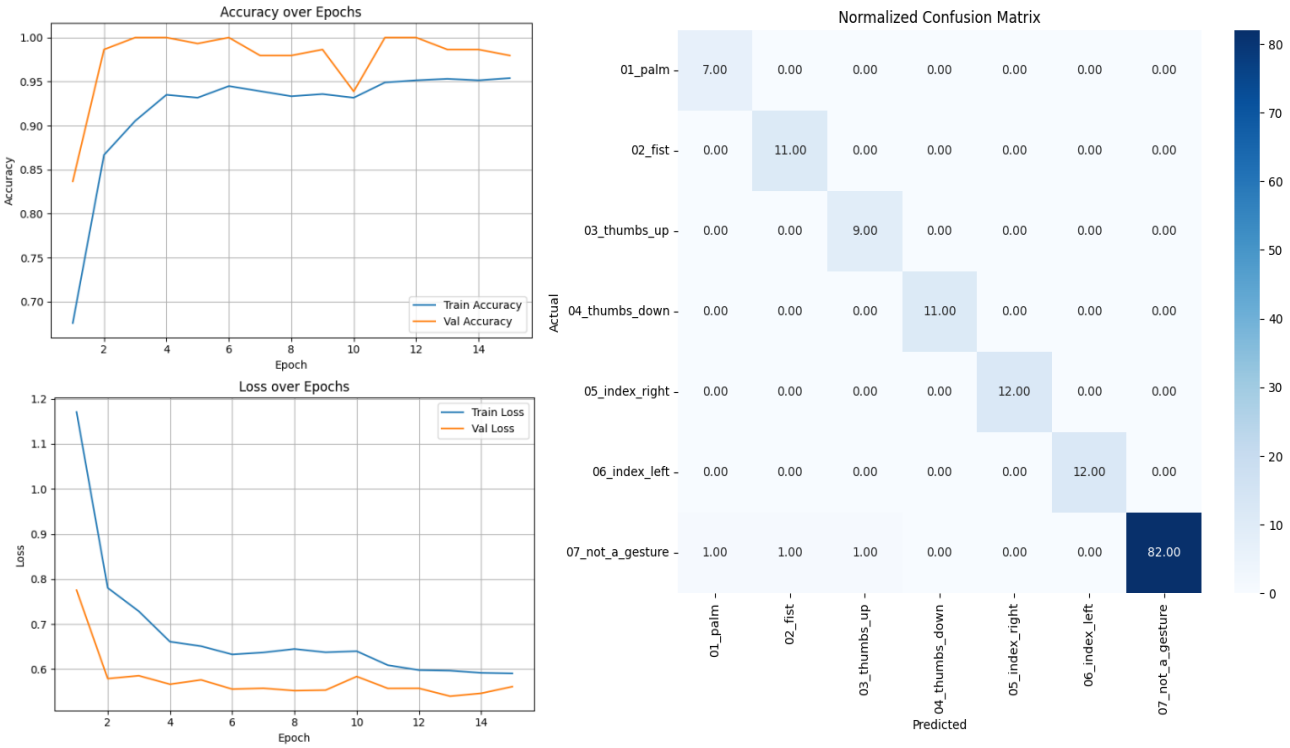


Figure 10: Model Evaluation

Outcome

- Achieved 97.9% validation accuracy with strong generalization
- Reached near-perfect classification across gesture classes
- Successfully handled ambiguous inputs via a dedicated “not-a-gesture” class
- Demonstrated stable training with minimal overfitting

Model setup

```python
from torchvision import models
import torch.nn as nn
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = models.resnet18(pretrained=True)
for param in model.parameters():
    param.requires_grad = False

# Fine-tune last block
for param in model.layer4.parameters():
    param.requires_grad = True

# Replace FC layer
model.fc = nn.Sequential(
    nn.Dropout(0.4),
    nn.Linear(model.fc.in_features, len(class_names))
)
model = model.to(device)
```

✓ 0.1s

Defining Loss, Optimizier and scheduler

```python
import torch.optim as optim
from torch.optim.lr_scheduler import ReduceLROnPlateau

criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

optimizer = optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3}
])

scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=2, verbose=True)
```

✓ 0.0s

Predicted: 06_index_left | Actual: 06_index_left



Figure 11: Model setup and training

Ball Joint Sensor Integration (Bachelor Thesis finished February 2024)

Investigation of methods for extracting rotational data from a mechanical ball joint system, where direct measurement of joint orientation is not inherently available. The project focused on enabling accurate and reliable sensing for use in control and monitoring applications.

Role

Developed a complete sensing approach for capturing and interpreting rotational motion from the ball joint. This included sensor selection, integration into the mechanical system, and development of data processing methods to extract meaningful rotational information.

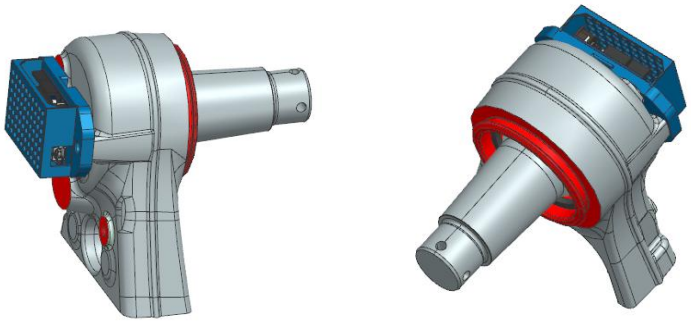


Figure 12 & 13: Joint full assembly

Technical Contributions

- Integrated sensors into a constrained mechanical ball joint assembly
- Developed signal processing methods to convert raw sensor data into usable rotational information
- Performed mechanical analysis to understand system behavior and measurement limitations
- Implemented data acquisition and processing workflows for validation
- Conducted experimental testing and calibration to improve accuracy and reliability
- Technologies: Sensors, Signal Processing, Data Acquisition, MATLAB/Python, Mechanical Analysis

Outcome

Developed a method for accurately extracting rotational information from an indirectly measurable system, enabling reliable sensing for control and monitoring applications in mechanically constrained environments. Demonstrated the integration of sensing, mechanical analysis, and signal processing for indirect state estimation.

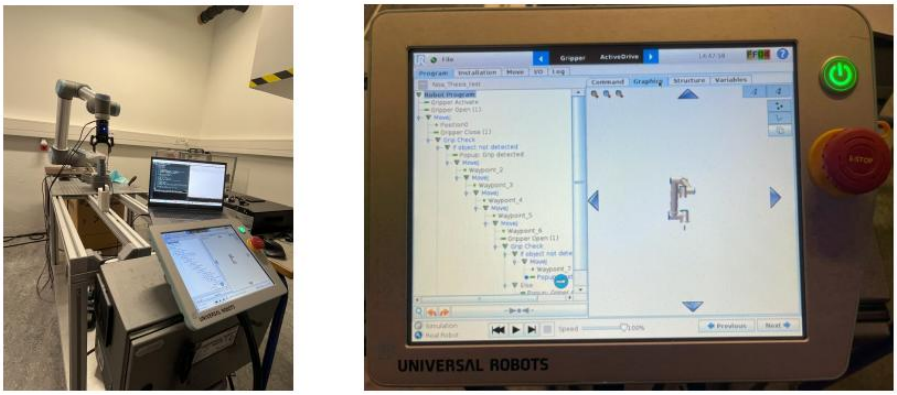


Figure 14 & 15: Testing setup

Joystick Test System – Danfoss Power Solutions

Development of hardware-based testing systems for validation of industrial joystick performance in real-world applications. The project focused on improving the efficiency, repeatability, and reliability of validation processes by transitioning from manual testing to automated and semi-automated test setups.

Role

Designed and built multiple test setups for joystick validation, contributing to both hardware development and the automation of testing procedures. Worked across the full development cycle, including concept development, mechanical design, system integration, and experimental validation. Collaborated with engineers to ensure test systems met functional requirements and aligned with product development needs.

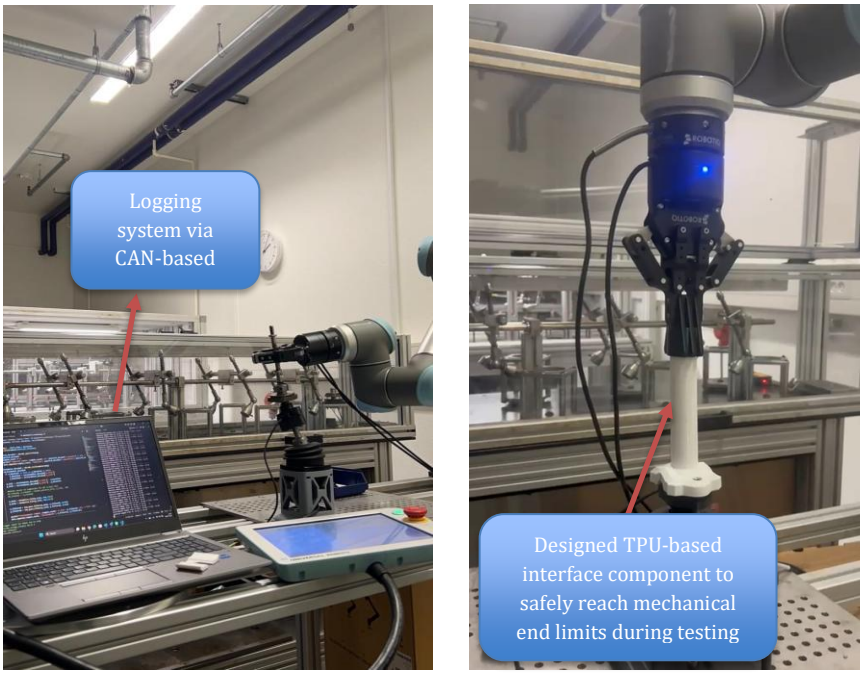


Figure 16 & 17: Test Setup

Technical Contributions

- Designed mechanical test rigs and fixtures using CAD (NX / Fusion)
- Integrated sensors and measurement systems for capturing joystick performance data using a MEAS position sensor
- Developed data logging and analysis workflows for evaluating system behavior via CAN-based communication
- Implemented robot-assisted testing for repeatable and automated validation sequences
- Conducted experimental validation and debugging to ensure accuracy and robustness of test setups

- Technologies: CAD (NX/Fusion), CAN Communication, Embedded Systems, Sensors, Python, Robotics

Outcome

Developed automated and semi-automated test systems for joystick validation, improving repeatability, measurement accuracy, and efficiency compared to manual testing processes. Enabled reliable data-driven validation through integrated sensing and CAN-based data acquisition.

Core skills

- ❖ Computer Vision (YOLOv8, OpenCV, stereo depth)
- ❖ Robot Programming (UR3, URScript, RTDE)
- ❖ Machine Learning (PyTorch, CNNs, transfer learning)
- ❖ CAN-bus integration & signal processing
- ❖ Mechanical design (NX, Fusion 360, FDM/TPU)
- ❖ Python · MATLAB · Embedded Systems (Arduino)